第9章

结构型设计模式

陈龙彪
longbiaochen@xmu.edu.cn

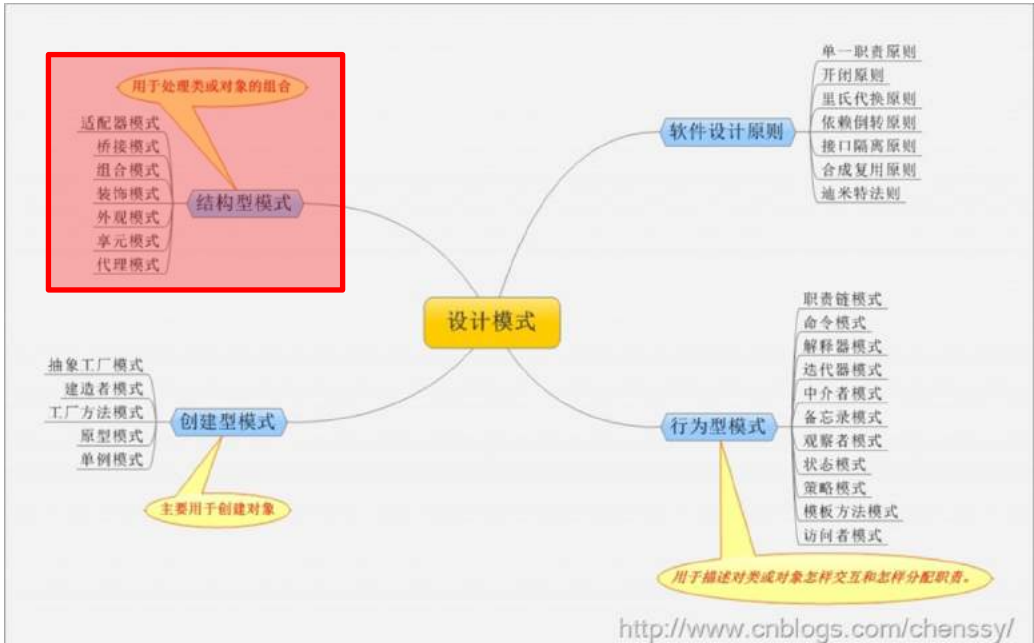
厦门大学信息学院
2019年4月

设计模式

范围\目的	创建型模式	结构型模式	行为型模式
类模式	工厂方法模式	（类）适配器模式	解释器模式 模板方法模式
对象模式	抽象工厂模式 建造者模式 原型模式 单例模式	（对象）适配器模式 桥接模式 组合模式 装饰模式 外观模式 享元模式 代理模式	职责链模式 命令模式 迭代器模式 中介者模式 备忘录模式 观察者模式 状态模式 策略模式 访问者模式

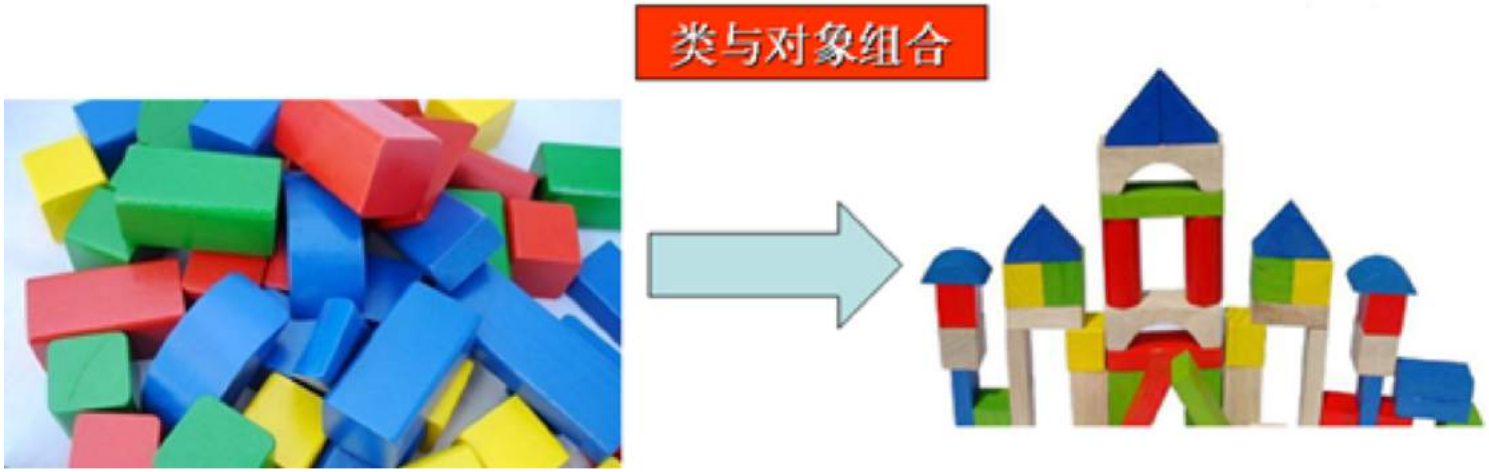
本章教学内容

- 结构型模式
  - 适配器模式（Adapter）
  - 桥接模式（Bridge）
  - 组合模式（Composite）
  - 装饰模式（Decorator）
  - 外观模式（Facade）
  - 享元模式（Flyweight）
  - 代理模式（Proxy）



结构型模式

- 结构型模式概述
  - 结构型模式（Structural Pattern）描述如何将类或者对象结合在一起形成更大的结构，就像搭积木，可以通过简单积木的组合形成复杂的、功能更为强大的结构



结构型模式

结构型模式概述

- 结构型模式可以分为类结构型模式和对象结构型模式
  - **类结构型模式**关心类的组合，由多个类可以组合成一个更大的系统，在类结构型模式中一般只存在继承关系和实现关系
  - **对象结构型模式**关心类与对象的组合，通过关联关系在一个类中定义另一个类的实例对象，然后通过该对象调用其方法。
  - 根据“合成复用原则”，在系统中尽量使用关联关系来替代继承关系，因此大部分结构型模式都是对象结构型模式

合成复用原则就是在一个新的对象里通过关联关系来使用一些已有的对象，使之成为新对象的一部分；新对象通过委派调用已有对象的方法达到复用功能的目的。复用时要尽量使用组合/聚合关系（关联关系），少用继承。



本章教学内容

- 结构型模式
  - 适配器模式（Adapter）
  - 桥接模式（Bridge）
  - 组合模式（Composite）
  - 装饰模式（Decorator）
  - 外观模式（Facade）
  - 享元模式（Flyweight）
  - 代理模式（Proxy）



适配器模式

- 模式动机
  - 在软件开发中采用类似于电源适配器的设计和编码技巧被称为适配器模式



适配器模式



适配器模式

模式动机

- 在适配器模式中可以定义一个包装类来包装不兼容接口的对象，这个包装类指的就是适配器（Adapter），它所包装的对象就是适配者（Adaptee），即被适配的类
- 适配器提供客户类需要的接口，适配器的实现就是把客户类的请求转化为对适配者的相应接口的调用
- 当客户类调用适配器的方法时，在适配器类的内部将调用适配者类的方法，而这个过程对客户类是透明的，客户类并不直接访问适配者类

适配器模式

- 模式定义
  - 适配器模式（Adapter Pattern）：将一个接口转换成客户希望的另一个接口，适配器模式使接口不兼容的那些类可以一起工作，其别名为包装器（Wrapper）
  - 适配器模式既可以作为类结构型模式，也可以作为对象结构型模式

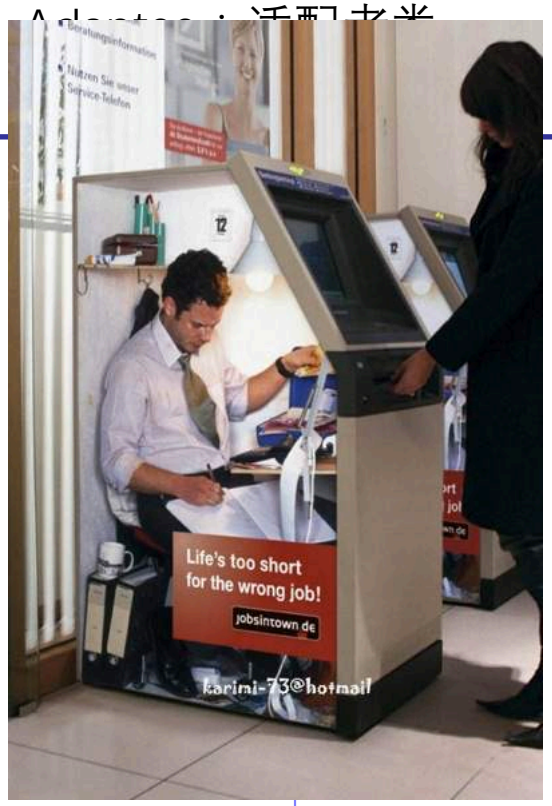
适配器模式

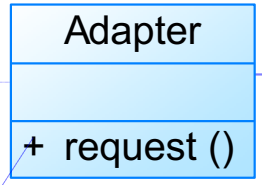
适配器模式包含如下角色：

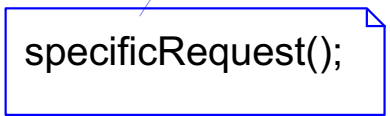
- Target：目标抽象类
- Adapter：适配器类
- Adaptee：适配者类
-

模式结构





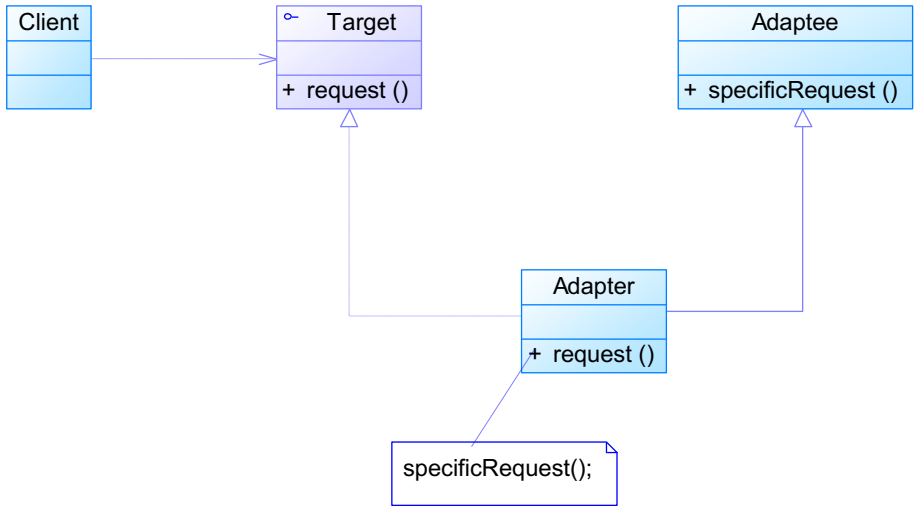




适配器模式

- 模式分析
  - 典型的类适配器代码

```
public class Adapter extends Adaptee implements Target {
     public void request() {
          specificRequest();
     }
}
```



适配器模式

- 模式分析
  - 典型的对象适配器代码

```
public class Adapter extends Target {
     private Adaptee adaptee;

     public Adapter(Adaptee adaptee) {
          this.adaptee = adaptee;
     }

     public void request() {
          adaptee.specificRequest();
     }
}
```

适配器模式

- 适配器模式实例与解析
  - 实例一：仿生机器人
    - 现需要设计一个可以模拟各种动物行为的机器人，在机器人中定义了一系列方法，如机器人叫喊方法cry、机器人移动方法move等
    - 如果希望在不修改已有代码的基础上使得机器人能够像狗一样叫和跑。请使用适配器模式进行系统设计





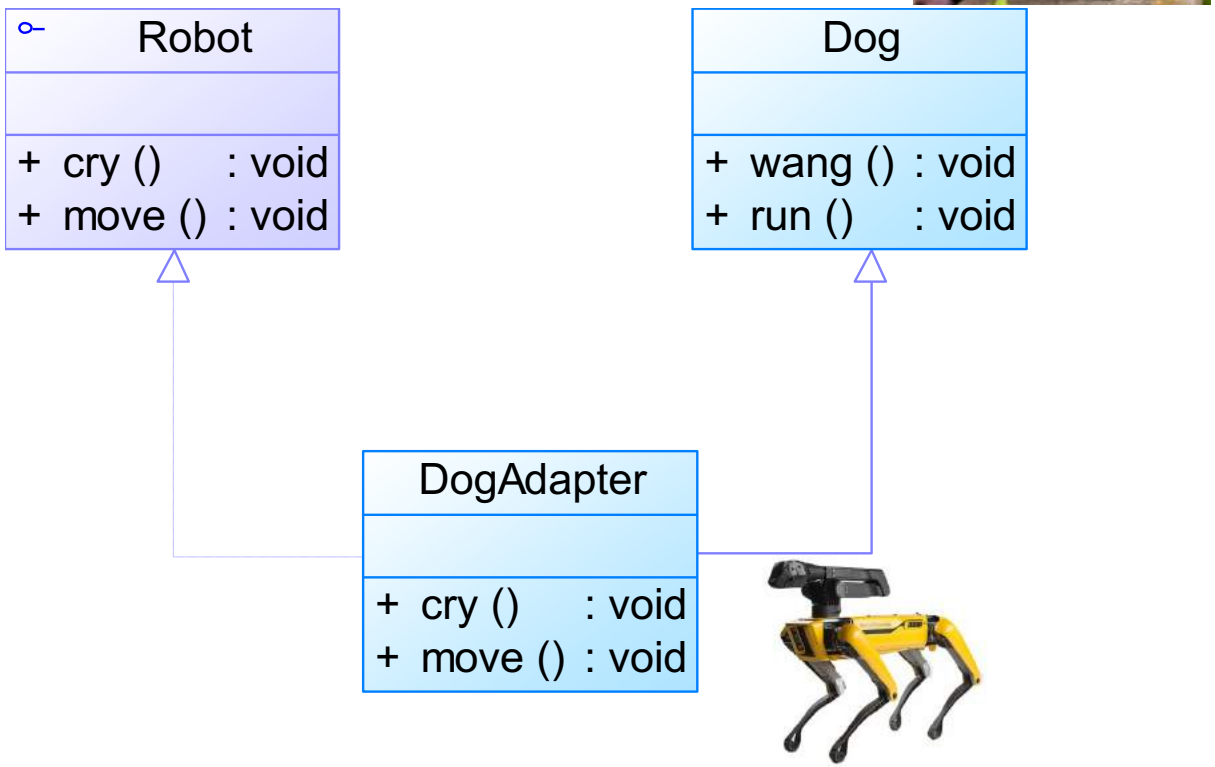


适配器模式

- 适配器模式实例与解析
  - 实例一：仿生机器人

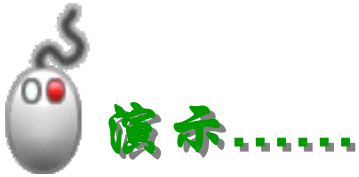






适配器模式

- 适配器模式实例与解析
  - 实例：仿生机器人
    - 参考代码（adapter）



适配器模式

模式优缺点

适配器模式的优点

- 将目标类和适配者类解耦：通过引入一个适配器类来重用现有的适配者类，而无须修改原有代码
- 增加了类的透明性和复用性：将具体的实现封装在适配者类中，对于客户端类来说是透明的，而且提高了适配者的复用性
- 灵活性和扩展性都非常好：通过使用配置文件，可以很方便地更换适配器，也可以在不修改原有代码的基础上增加新的适配器类，完全符合“开闭原则”

适配器模式

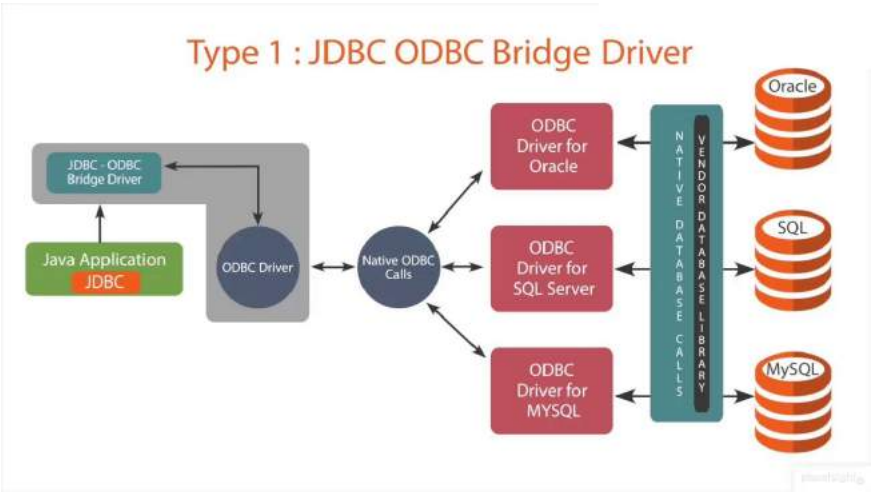
- 模式优缺点
  - 类适配器模式的缺点如下：
    - 对于Java、C#等不支持多重继承的语言，一次最多只能适配一个适配者类
    - 目标类只能为抽象类，不能为具体类，其使用有一定的局限性，如不能将一个适配者类和它的子类都适配到目标接口

适配器模式

- 模式适用环境
  - 在以下情况下可以使用适配器模式：
    - 系统需要使用现有的类，而这些类的接口不符合系统的需要（既有代码）
    - 想要建立一个可以重复使用的类，用于与一些彼此之间没有太大关联的一些类，包括一些可能在将来引进的类一起工作

适配器模式

- 模式应用
  - Java数据库连接工具JDBC使得Java语言程序能够与数据库连接，并使用SQL语言来查询和操作数据
  - JDBC给出一个客户端通用的抽象接口，每一个具体的数据库引擎（如SQL Server、Oracle、MySQL等）的JDBC驱动软件都是一个介于JDBC接口和数据库引擎接口之间的适配器软件
  - 抽象的JDBC接口和各个数据库引擎API之间都需要相应的适配器软件，这就是为各个不同数据库引擎准备的驱动程序



本章教学内容

- 结构型模式
  - 适配器模式（Adapter）
  - 桥接模式（Bridge）
  - 组合模式（Composite）
  - 装饰模式（Decorator）
  - 外观模式（Facade）
  - 享元模式（Flyweight）
  - 代理模式（Proxy）



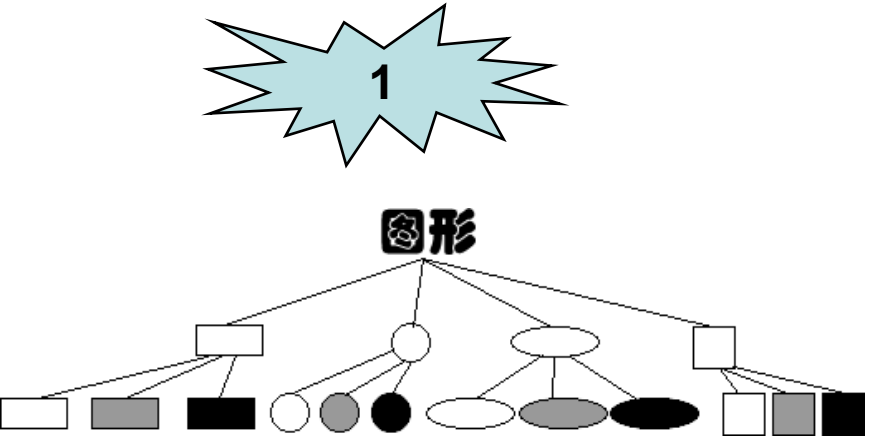
桥接模式

模式动机

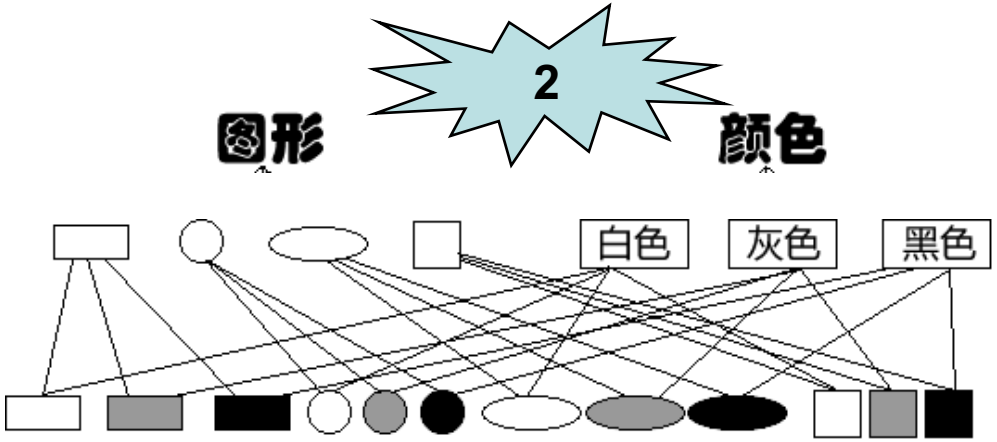
- 设想如果要绘制矩形、圆形、椭圆、正方形，我们至少需要4个形状类，但是如果绘制的图形需要具有不同的颜色，如红色、绿色、蓝色等，此时有两种设计方案：
  - 第一种设计方案是为每一种形状都提供一套各种颜色的版本
  - 第二种设计方案是根据实际需要对形状和颜色进行组合

桥接模式

模式动机



继承关系



关联关系

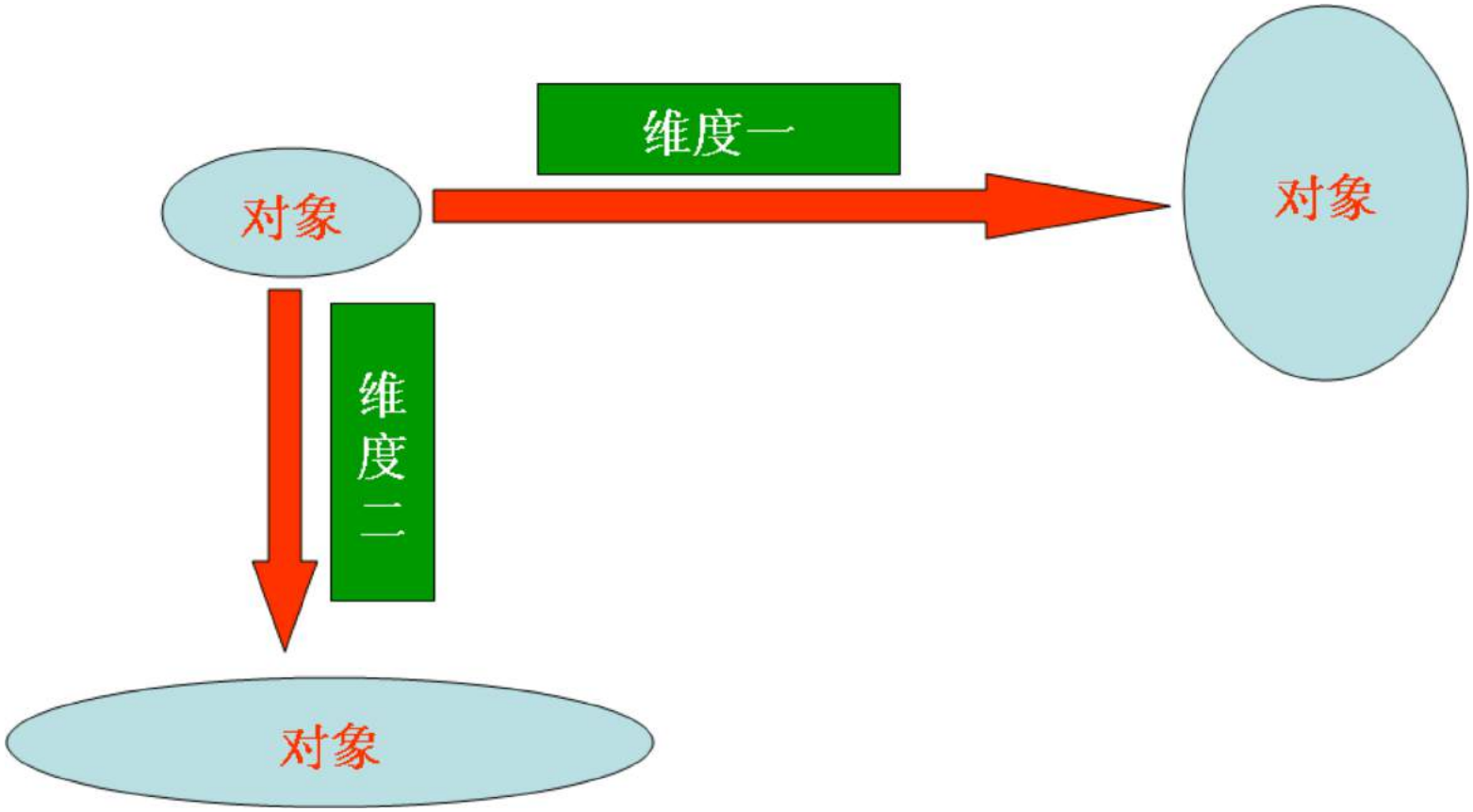
桥接模式

模式动机

- 对于有两个变化维度（原因）的系统，采用方案二来进行设计系统中类的个数更少，且系统扩展更为方便
- 设计方案二即是桥接模式。桥接模式将继承关系转换为关联关系，从而降低了类与类之间的耦合，减少了代码编写量

桥接模式

模式动机



桥接模式

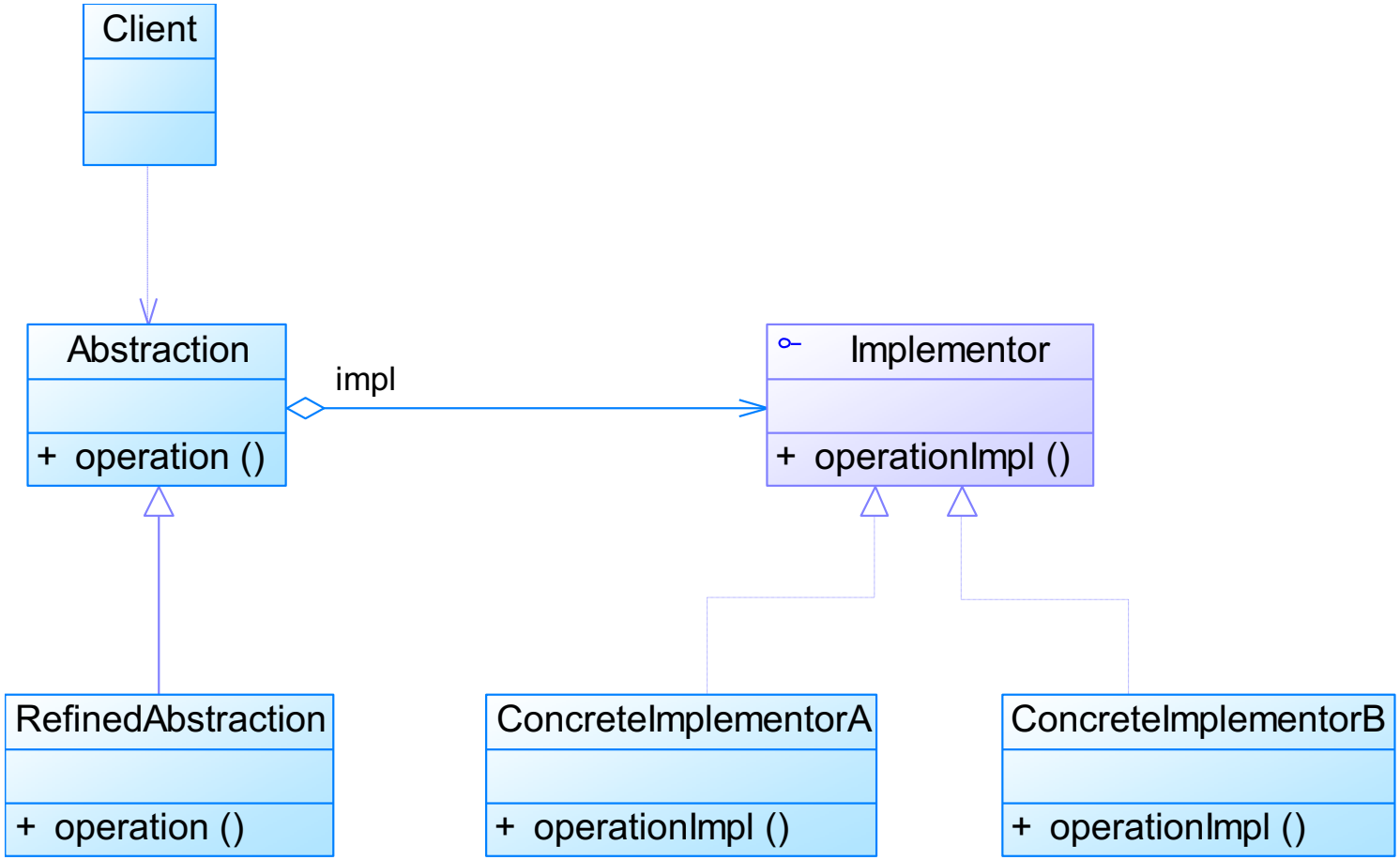
- 模式定义
  - 桥接模式（Bridge Pattern）：将抽象部分与它的实现部分分离，使它们都可以独立地变化
  - 桥接模式是一种对象结构型模式

桥接模式

桥接模式包含如下角色：

- Abstraction：抽象类
- RefinedAbstraction：扩充抽象类
- Implementor：实现类接口
- ConcreteImplementor：具体实现类

模式结构



桥接模式

- 模式分析
  - 典型的实现类接口代码：

```
public interface Implementor {
     public void operationImpl();
}
```

```
public abstract class Abstraction {
     protected Implementor impl;

     public void setImpl(Implementor impl) {
          this.impl = impl;
     }

     public abstract void operation();
}
```

桥接模式

- 模式分析
  - 典型的扩充抽象类代码：

```
public abstract class Abstraction {
    protected Implementor impl;

    public void setImpl(Implementor impl) {
        this.impl = impl;
    }

    public abstract void operation();
}
```

桥接模式

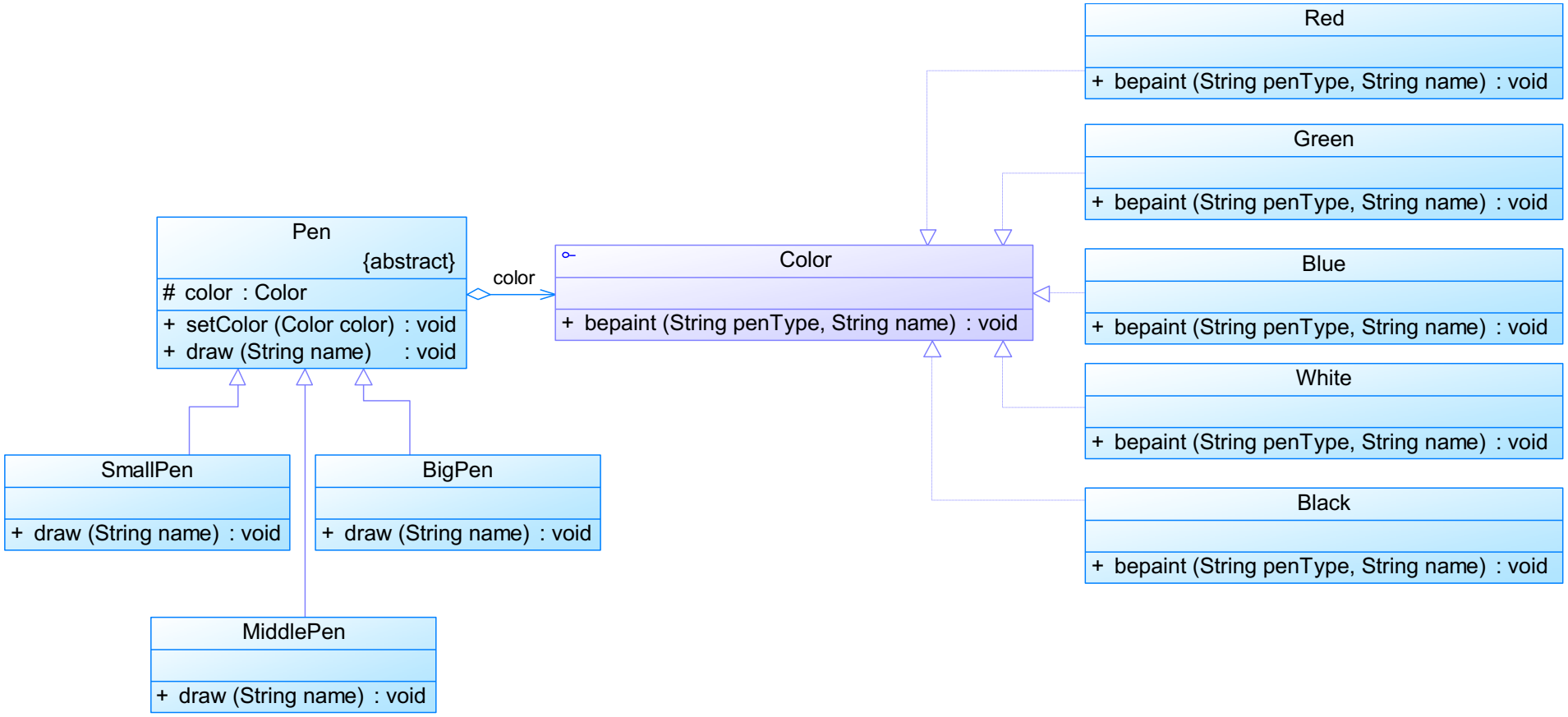
- 模式分析
  - 理解桥接模式，重点需要理解如何将抽象化（Abstraction）与实现化（Implementation）脱耦，使得二者可以独立地变化。
    - 抽象化：抽象化就是忽略一些信息，把不同的实体当作同样的实体对待。在面向对象中，将对象的共同性质抽取出来形成类的过程即为抽象化的过程。
    - 实现化：针对抽象化给出的具体实现，就是实现化，抽象化与实现化是一对互逆的概念，实现化产生的对象比抽象化更具体，是对抽象化事物的进一步具体化的产物。
    - 脱耦：脱耦就是将抽象化和实现化之间的耦合解脱开，或者说是将它们之间的强关联改换成弱关联，将两个角色之间的继承关系改为关联关系。桥接模式中的所谓脱耦，就是指在一个软件系统的抽象化和实现化之间使用关联关系（组合或者聚合关系）而不是继承关系，从而使两者可以相对独立地变化，这就是桥接模式的用意。

桥接模式

- 桥接模式实例与解析
  - 实例：模拟毛笔
    - 现需要提供大中小3种型号的画笔，能够绘制5种不同颜色，如果使用蜡笔，我们需要准备3*5=15支蜡笔，也就是说必须准备15个具体的蜡笔类
    - 而如果使用毛笔的话，只需要3种型号的毛笔，外加5个颜料盒，用3+5=8个类就可以实现15支蜡笔的功能
    - 使用桥接模式来模拟毛笔的使用过程

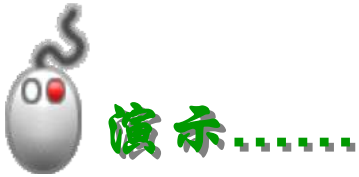
桥接模式

- 桥接模式实例与解析
  - 实例一：模拟毛笔



桥接模式

- 桥接模式实例与解析
  - 实例：模拟毛笔
    - 参考代码（bridge）



桥接模式

- 桥接模式实例与解析
  - 实例：倚天剑和屠龙









桥接模式

模式优缺点

桥接模式的优点

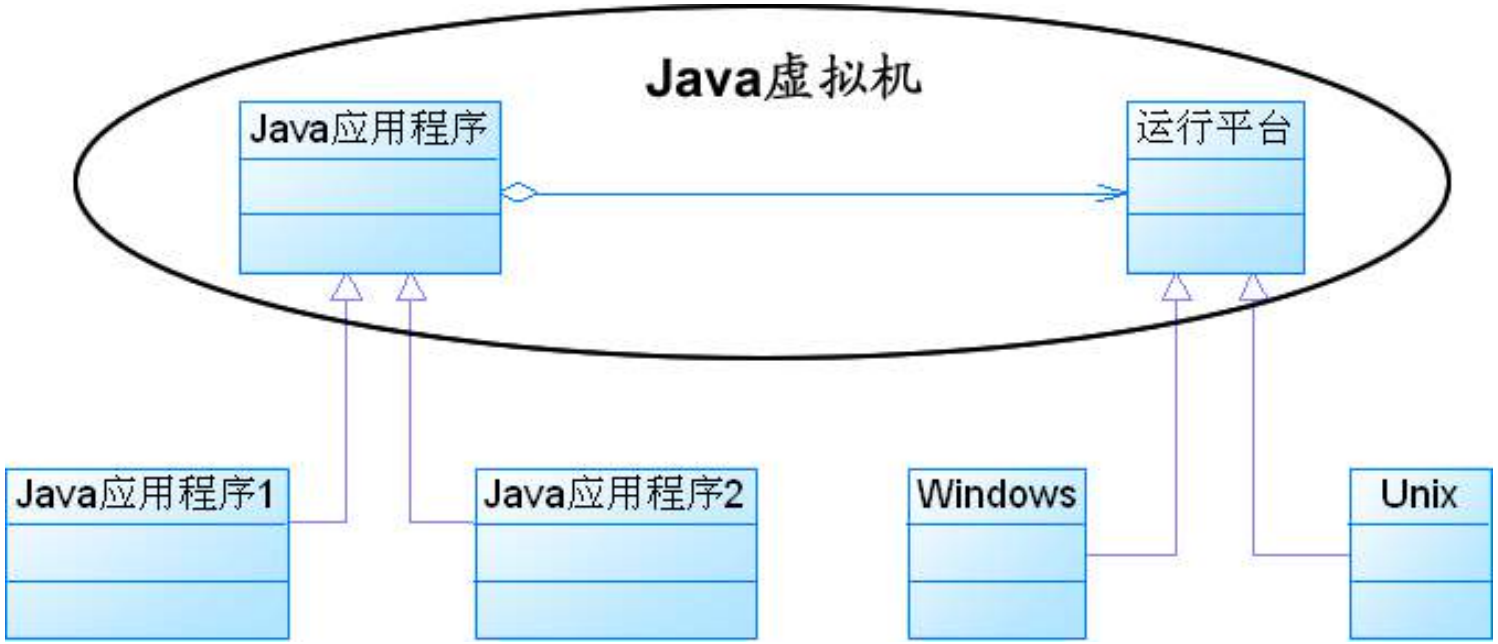
- 分离抽象接口及其实现部分
- 桥接模式有时类似于多继承方案，但是多继承方案违背了类的单一职责原则（即一个类只有一个变化的原因），复用性比较差，而且多继承结构中类的个数非常庞大，桥接模式是比多继承方案更好的解决方法
- 桥接模式提高了系统的可扩充性，在两个变化维度中任意扩展一个维度，都不需要修改原有系统
- 实现细节对客户透明，可以对用户隐藏实现细节

桥接模式

- 模式优缺点
  - 桥接模式的缺点
    - 桥接模式的引入会增加系统的理解与设计难度，由于聚合关联关系建立在抽象层，要求开发者针对抽象进行设计与编程
    - 桥接模式要求正确识别出系统中两个独立变化的维度，因此其使用范围具有一定的局限性

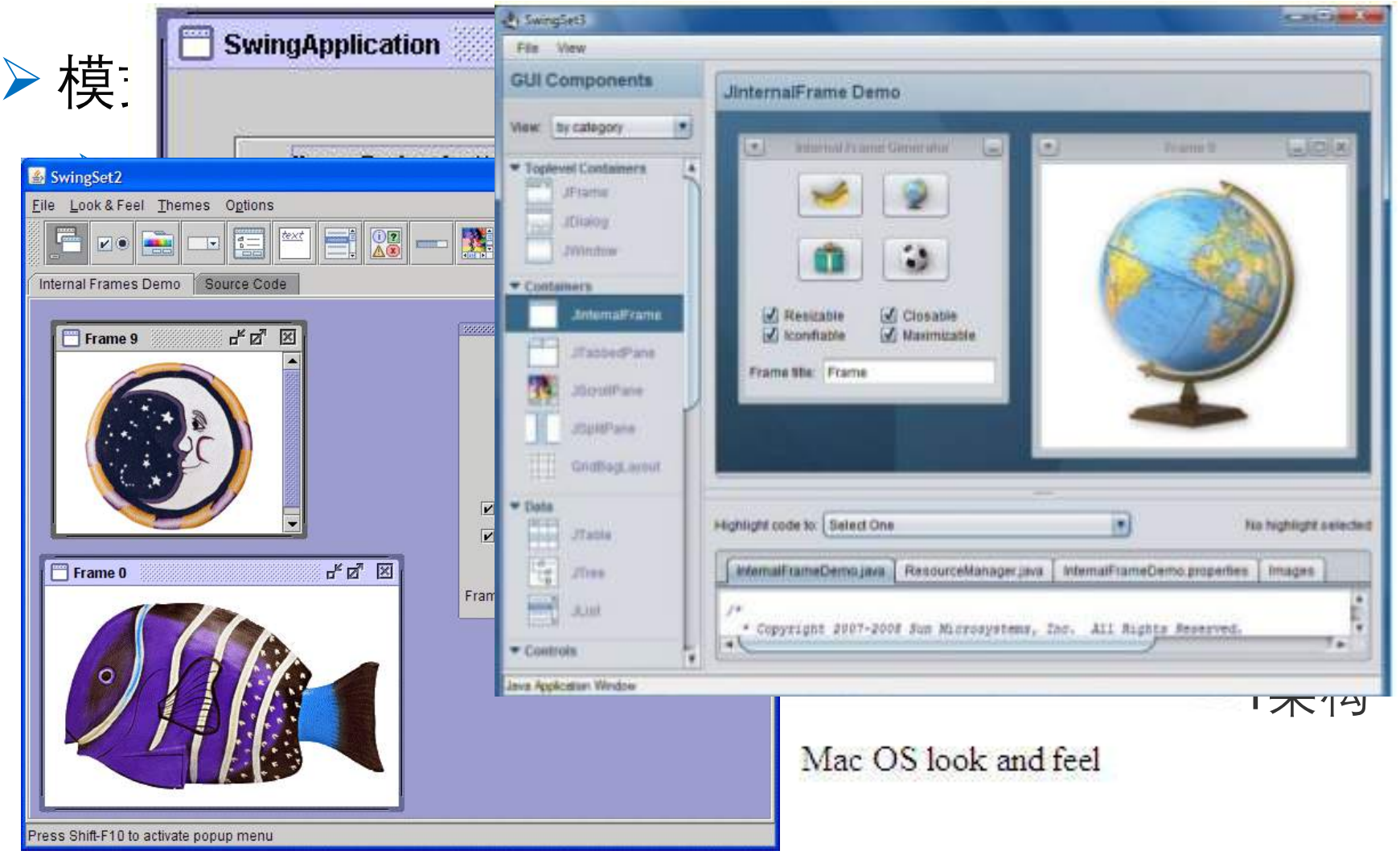
桥接模式

- 模式应用
  - (1) Java语言通过Java虚拟机实现了平台的无关性



桥接模式

- 模:


SwingApplication
SwingSet3
File View
GUI Components
JInternalFrame Demo
SwingSet2
File Look & Feel Themes Options
Internal Frames Demo
Source Code
Frame 9
Frame 0
Press Shift-F10 to activate popup menu
Mac OS look and feel

桥接模式

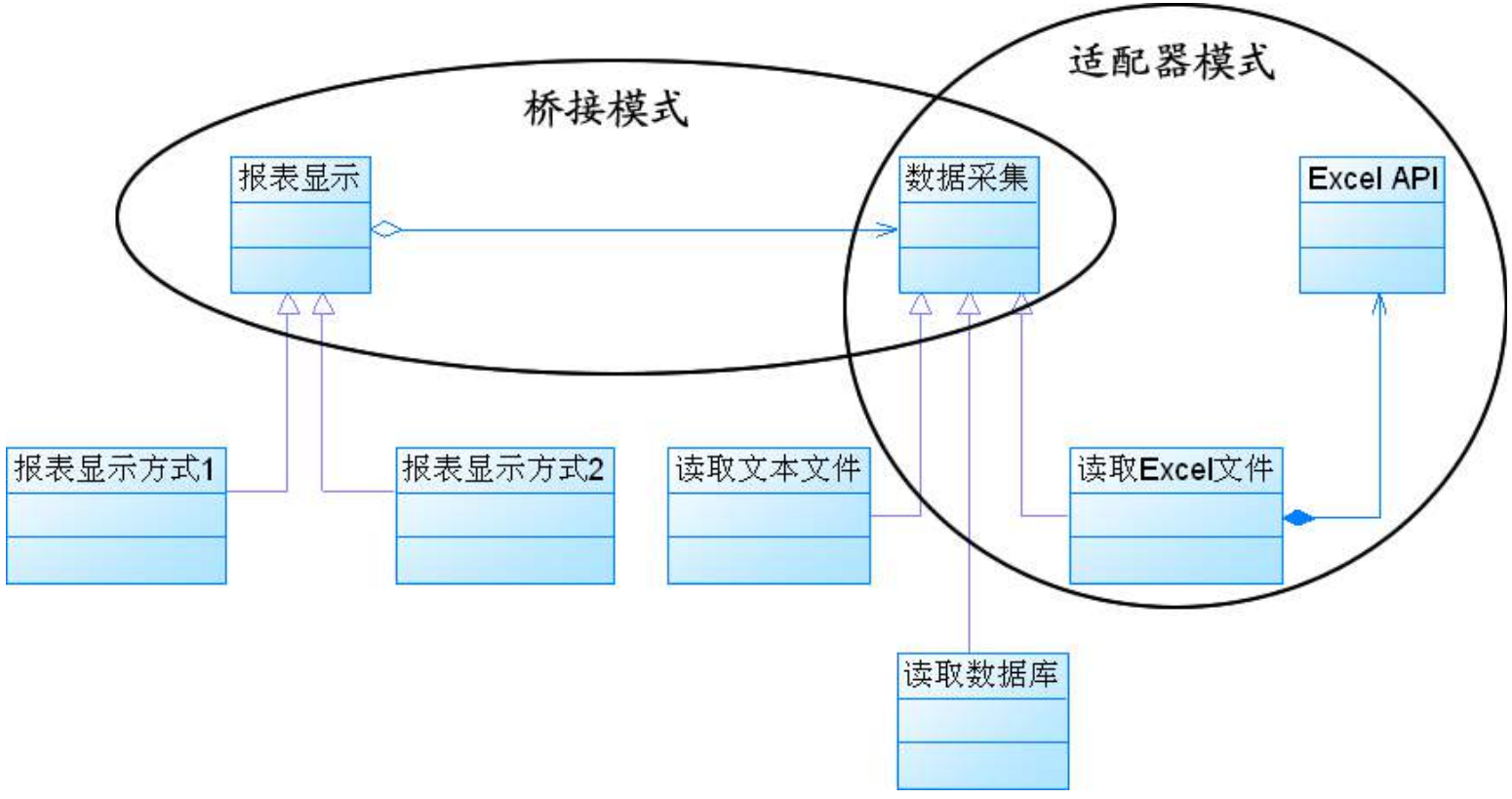
模式扩展

适配器模式与桥接模式的联用

- 桥接模式和适配器模式用于设计的不同阶段，桥接模式用于系统的初步设计，对于存在两个独立变化维度的类可以将其分为抽象化和实现化两个角色，使它们可以分别进行变化；而在初步设计完成之后，当发现系统与已有类无法协同工作时，可以采用适配器模式。但有时候在设计初期也需要考虑适配器模式，特别是那些涉及到大量第三方应用接口的情况

桥接模式

- 模式扩展
  - 适配器模式与桥接模式的联用



本章教学内容

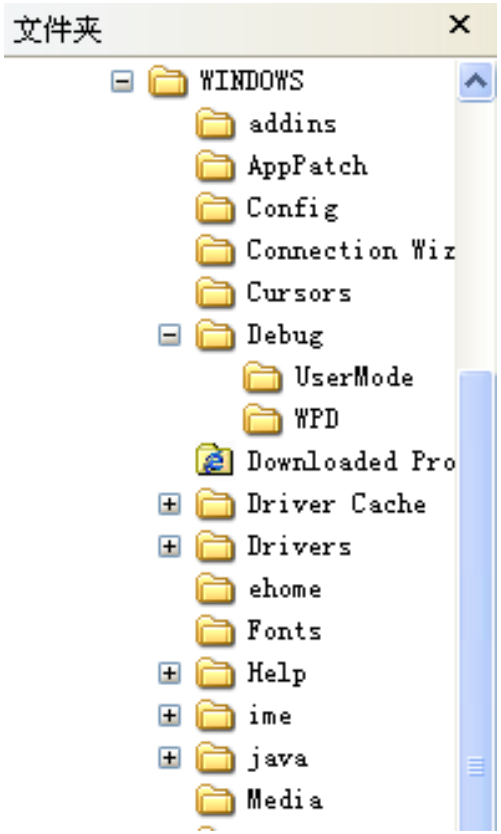
- 结构型模式
  - 适配器模式（Adapter）
  - 桥接模式（Bridge）
  - 组合模式（Composite）
  - 装饰模式（Decorator）
  - 外观模式（Facade）
  - 享元模式（Flyweight）
  - 代理模式（Proxy）

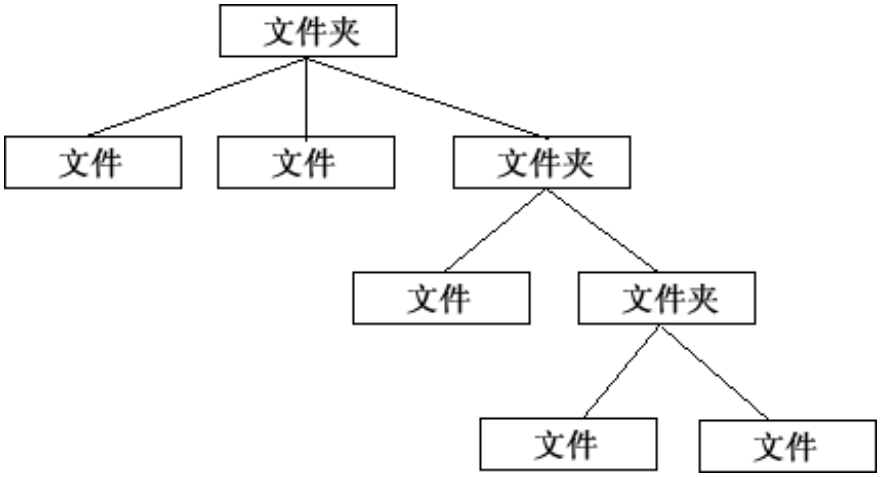


组合模式

模式动机

- 查找WINDOWS中以s结尾的文件和文件夹



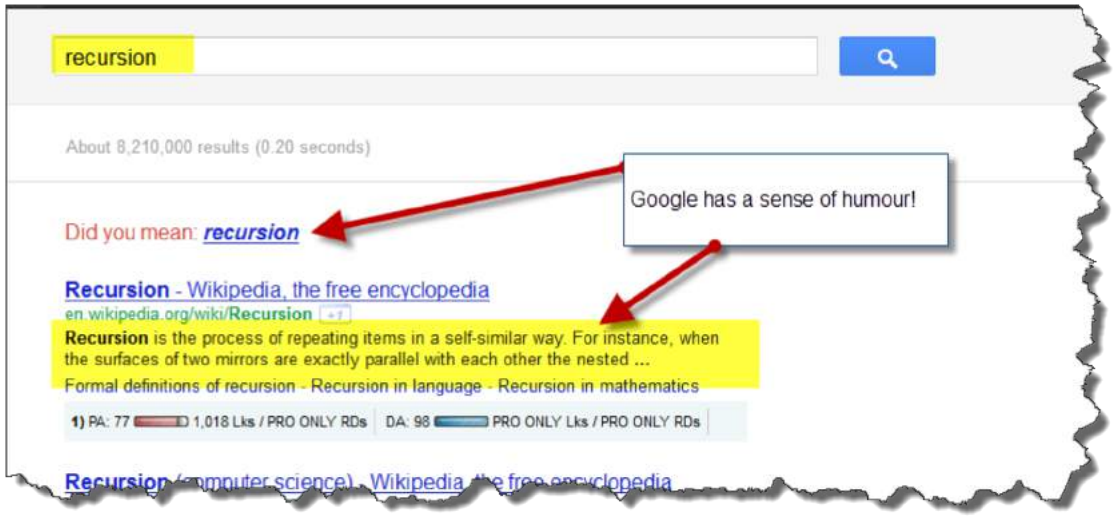


组合模式

- 模式动机
  - 对于树形结构，当容器对象（如文件夹）的某一个方法被调用时，将遍历整个树形结构，寻找也包含这个方法的成员对象（可以是容器对象，也可以是叶子对象，如子文件夹和文件）并调用执行（递归调用）
  - 由于容器对象和叶子对象在功能上的区别，在使用这些对象的客户端代码中必须有区别地对待容器对象和叶子对象，而实际上大多数情况下客户端希望一致地处理它们，因为对这些对象区别对待将会使得程序非常复杂

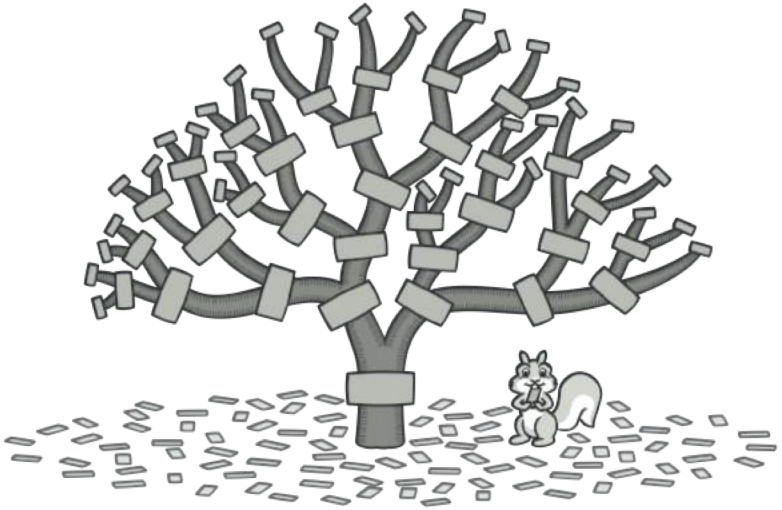
组合模式

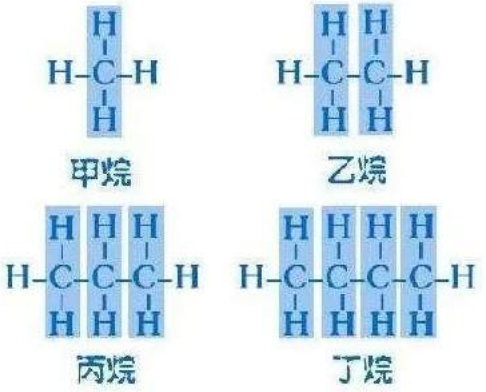
- 模式动机
  - 组合模式将容器对象和叶子对象进行递归组合，使得用户在使用时无须对它们进行区分，可以一致地对待容器对象和叶子对象，这就是组合模式的模式动机



组合模式

- 模式定义
  - 组合模式（Composite Pattern）组合多个对象形成树形结构，以表示“整体-部分”的结构层次
  - 组合模式对单个对象（即叶子对象）和组合对象（即容器对象）的使用具有一致性



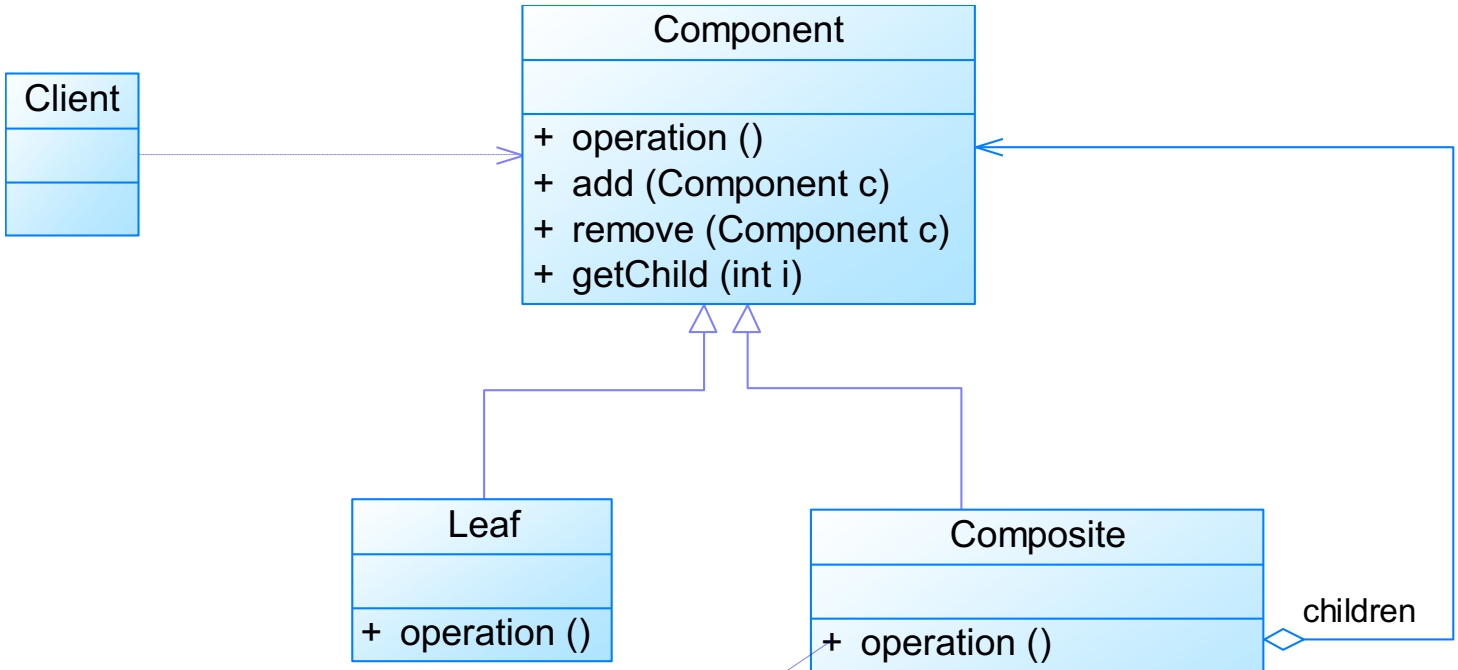


组合模式

组合模式包含如下角色：

- Component: 抽象构件
- Leaf: 叶子构件
- Composite: 容器构件
- Client: 客户类

模式结构

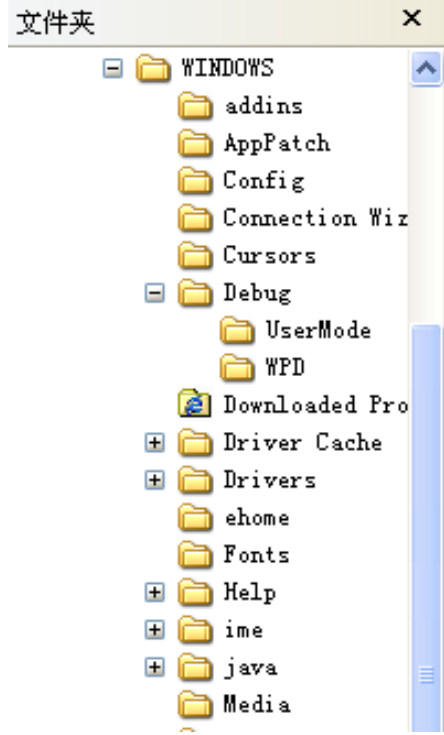


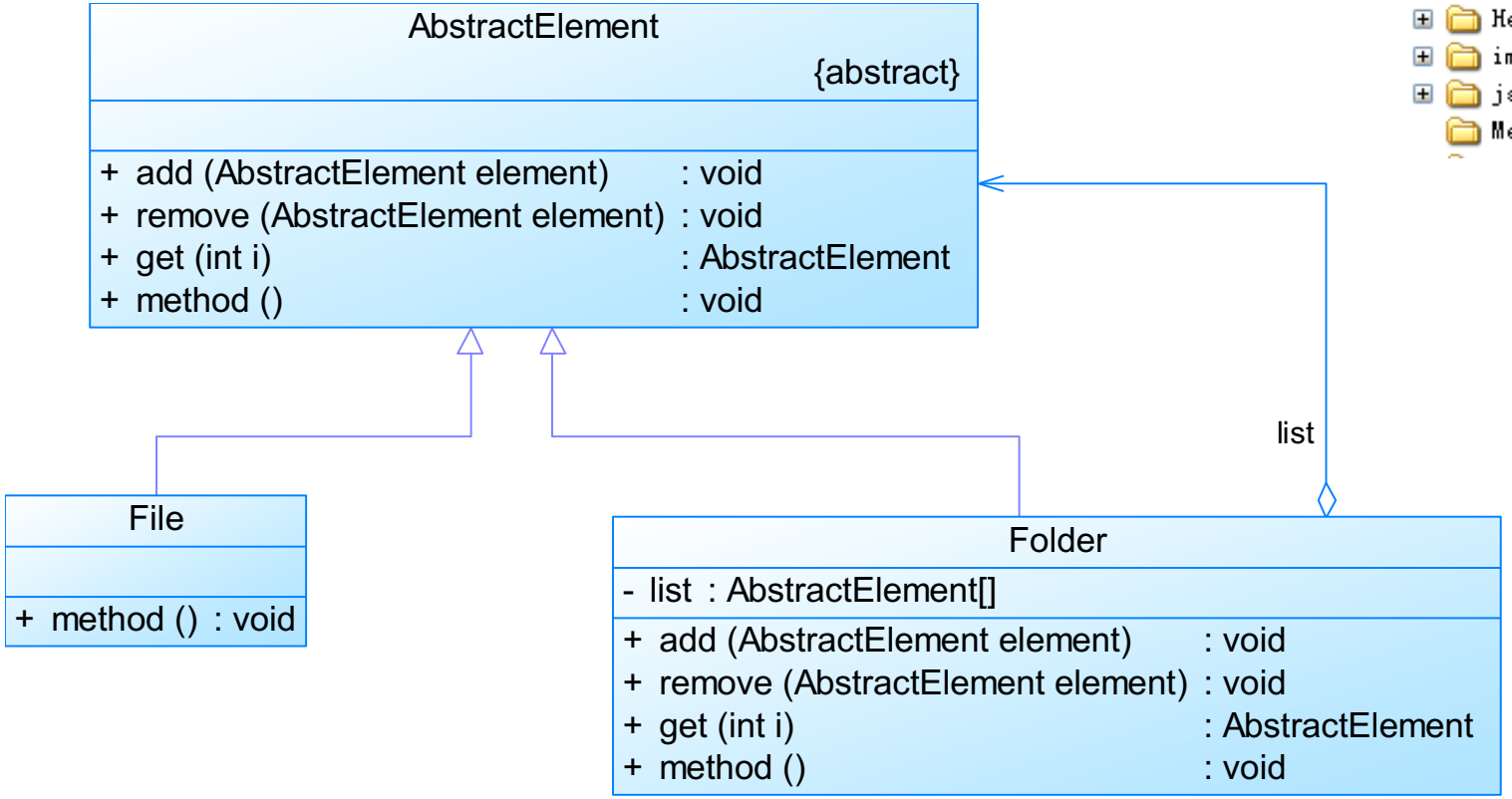
组合模式

- 模式分析
  - 组合模式的关键是定义了一个抽象构件类，它既可以代表叶子，又可以代表容器
  - 客户端：针对该抽象构件类进行编程，无须知道它到底表示的是叶子还是容器，可以对其进行统一处理
  - 容器对象：与抽象构件类之间还建立一个聚合关联关系，在容器对象中既可以包含叶子，也可以包含容器，以此实现递归组合，形成一个树形结构

组合模式

- 模式分析
  - 文件系统组合模式结构图：





组合模式

- 模式分析
  - 典型的抽象构件角色代码：

```
public abstract class Component {
    public abstract void add(Component c);
    public abstract void remove(Component c);
    public abstract Component getChild(int i);
    public abstract void operation();
}
```

组合模式

- 模式分析
  - 典型的容器构件角色代码：

```
public class Composite extends Component {
      private ArrayList list = new ArrayList();
      public void add(Component c) {
            list.add(c);
      }
      public void remove(Component c) {
            list.remove(c);
      }
      public Component getChild(int i) {
            (Component)list.get(i);
      }
      public void operation() {
            for(Object obj : list) {
                  ((Component)obj).operation();
            }
      }
}
```

组合模式

- 模式分析
  - 典型的叶子构件角色代码：

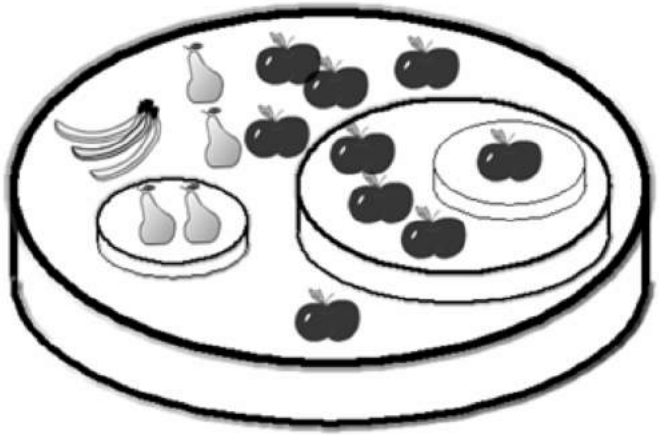
```
public class Leaf extends Component {
     public void add(Component c) {
          // Exception: forbidden operation
     }
     public void remove(Component c) {
          // Exception: forbidden operation
     }
     public Component getChild(int i) {
          // Exception: forbidden operation
     }
     public void operation() {
          // Implementation
     }
}
```

组合模式

组合模式实例与解析

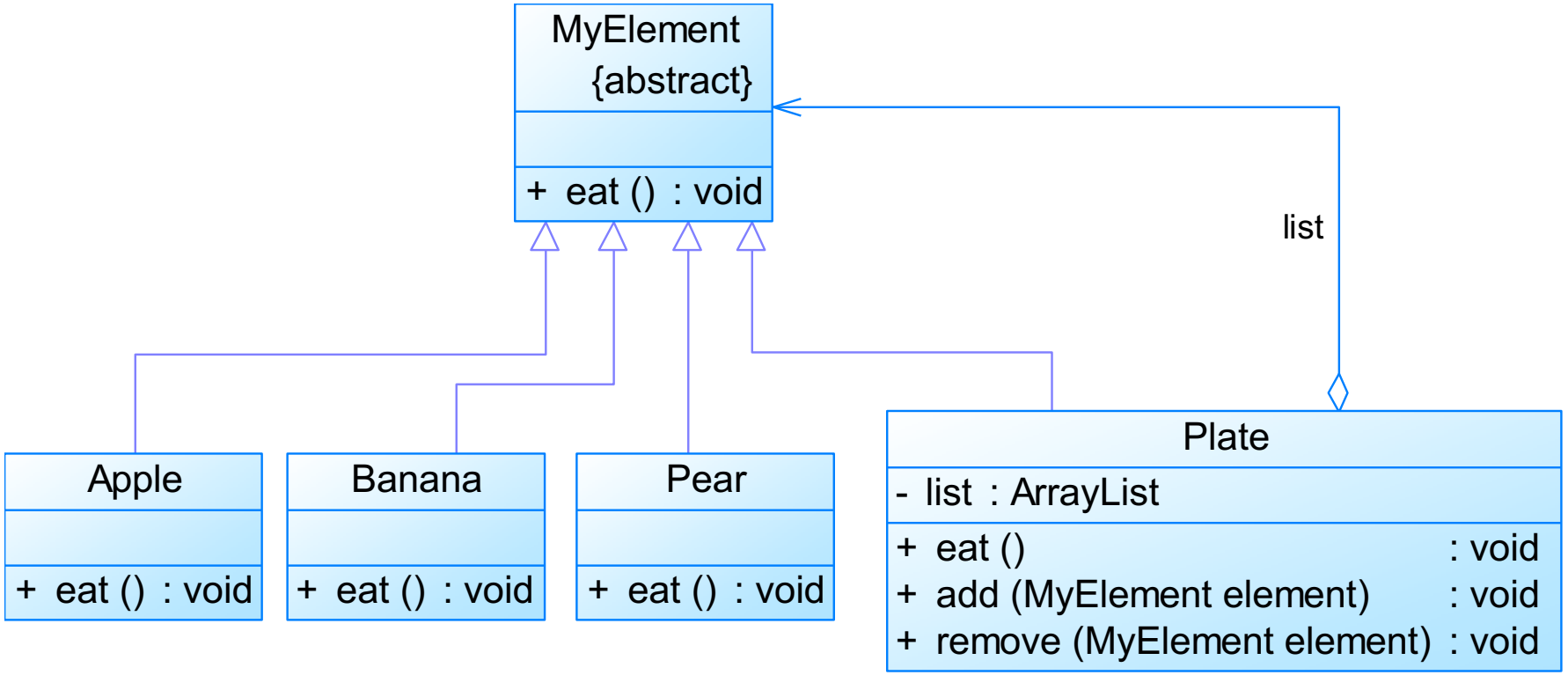
实例：水果盘

- 在水果盘(Plate)中有一些水果，如苹果(Apple)、香蕉(Banana)、梨子(Pear)，当然大水果盘中还可以有小水果盘，现需要对盘中的水果进行遍历（吃），当然如果对一个水果盘执行“吃”方法，实际上就是吃其中的水果。使用组合模式模拟该场景。



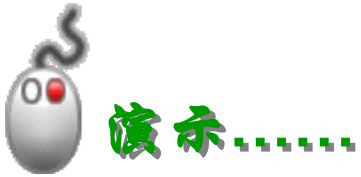
组合模式

- 组合模式实例与解析
  - 实例：水果盘



组合模式

- 组合模式实例与解析
  - 实例：水果盘
    - 参考代码 (Chapter 12 Composite\sample01)

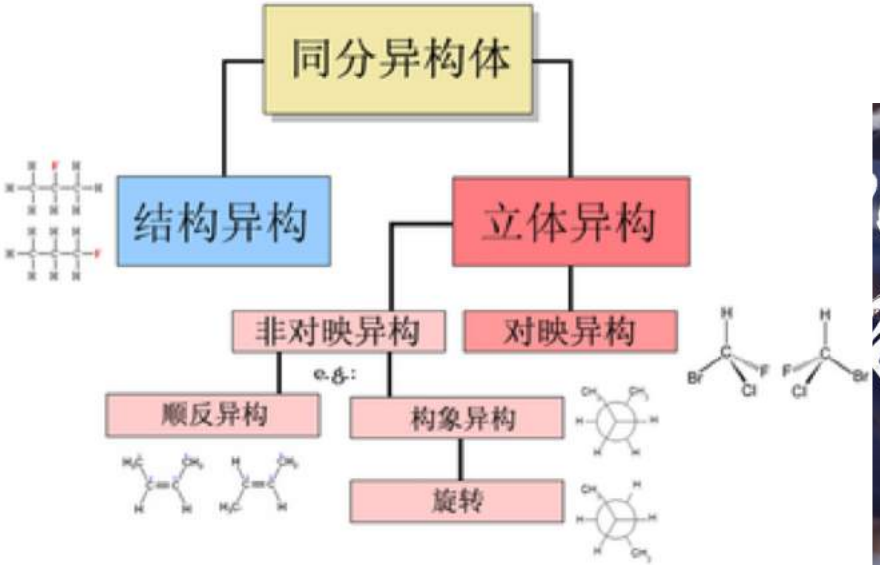


组合模式

- 模式优缺点
  - 组合模式的优点
    - 可以清楚地定义分层次的复杂对象，表示对象的全部或部分层次，使得增加新构件也更容易
    - 定义了包含叶子对象和容器对象的类层次结构，叶子对象可以被组合成更复杂的容器对象，而这个容器对象又可以被组合，这样不断递归下去，可以形成复杂的树形结构
    - 客户端调用简单，可以一致地使用组合结构或其中单个对象
    - 更容易在组合体内加入对象构件，客户端不必因为加入了新的对象构件而更改原有代码

组合模式

- 模式优缺点
  - 组合模式的缺点
    - 使设计变得更加抽象，对象的业务规则如果很复杂，则实现组合模式具有很大挑战性
    - 不是所有的方法都与叶子对象子类都有关联（异构性）
    - 增加新构件时可能会产生一些问题





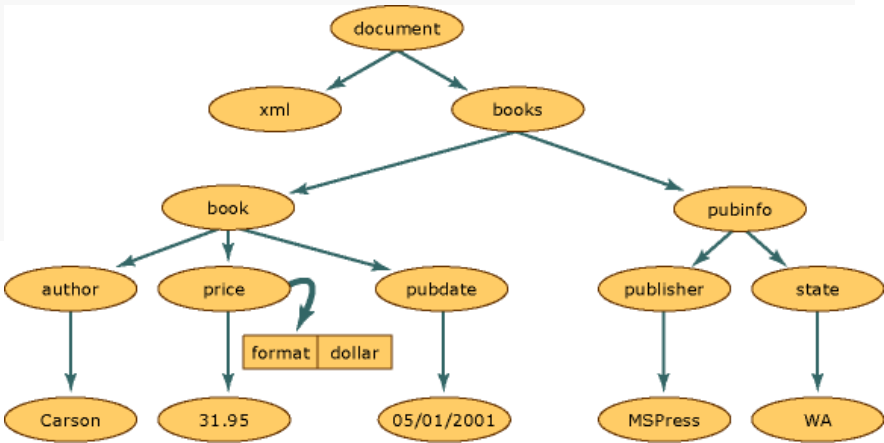
组合模式

- 模式适用环境
  - 在以下情况下可以使用组合模式：
    - 需要表示一个对象整体或部分层次，在具有整体和部分的层次结构中，希望通过一种方式忽略整体与部分的差异，可以一致地对待它们
    - 让客户能够忽略不同对象层次的变化，客户端可以针对抽象构件编程，无须关心对象层次结构的细节
    - 对象的结构是动态的并且复杂程度不一样，但客户需要一致地处理它们

组合模式

- 模式应用
  - （1） XML文档解析

```
<?xml version="1.0"?>
  <books>
    <book>
        <author>Carson</author>
        <price format="dollar">31.95</price>
        <pubdate>05/01/2001</pubdate>
    </book>
    <pubinfo>
        <publisher>MSPress</publisher>
        <state>WA</state>
    </pubinfo>
  </books>
```



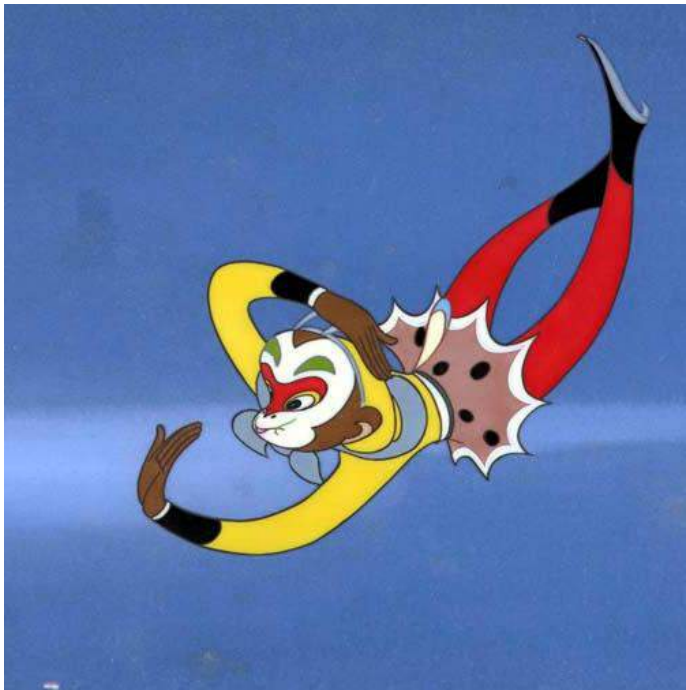
组合模式

- 模式应用
  - （2）操作系统中的目录结构是一个树形结构，因此在对文件和文件夹进行操作时可以应用组合模式，例如杀毒软件在查毒或杀毒时，既可以针对一个具体文件，也可以针对一个目录。如果是对目录查毒或杀毒，将递归处理目录中的每一个子目录和文件。



本章教学内容

- 结构型模式
  - 适配器模式（Adapter）
  - 桥接模式（Bridge）
  - 组合模式（Composite）
  - 装饰模式（Decorator）
  - 外观模式（Facade）
  - 享元模式（Flyweight）
  - 代理模式（Proxy）



装饰模式

模式动机

Lina 套装



Bewitching Flare
5 件待售，起价 $14.40



Fashion of the Scorching Princess Set
0件可购——发起买入订单



Inscribed Divine Flame
0件可购——发起买入订单



Smoldering Journey
0件可购——发起买入订单



Dragonfire
0件可购——发起买入订单



The Battle Caster Set
3 件待售，起价 $0.08



The Garments of the Charred Bloodline Set
2 件待售，起价 $0.09



Ember Crane Set
1 件待售，起价 $0.18

通常每年的6月10—15日（插秧后15天左右），在水稻田内放入小鸭子是最佳时间，一般1亩地放15只小鸭。买回的小鸭雏育10—15天左右，就可以放到水稻田里了。水稻田里都有网子圈着，小鸭子可以自由自在地在水稻田里畅游，一天需要喂两次，早一次，晚一次，稻田的边上有鸭舍，到了晚上，小鸭可以游到鸭舍中休息。

但在把小鸭放到稻田时，还要注意气温情况，如果气温低，就不能放，刚出生十几天的小鸭，抗冷能力还不是十分强，很可能冻坏，气温高时才能放入，像2013年的气温就不错，温度较高，可以准时放鸭。

只有无化肥和农药残留的有机生态环境才能保证小鸭安全长大，同时，鸭子在田间活动可以起到活水、除虫、除草的作用，水稻不会产生病虫害，无需使用农药。鸭的粪便作为肥料，可有效提高稻田的有机质含量，不用使用化肥，但需要适当施用一些酵素有机肥。

一直到水稻出穗，晚熟品种在秋季收获，也就是在此前的9月份时，就可以把小鸭子从稻田取出，因为到了水稻出穗时，鸭子会吃水稻的稻穗，而此时的水稻也无需鸭子的共生。不过，这时的小鸭已经不再是小鸭，而是长到了2—3斤的大鸭子了。

这个过程的关键在于，动物和植物同生共作，一年稻一批鸭，相辅相成，从而生产出绿色鸭产品和无农药残留的有机绿色大米。





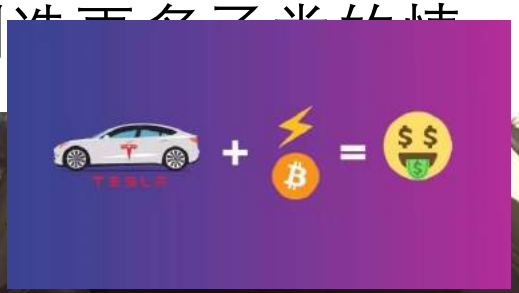
装饰模式

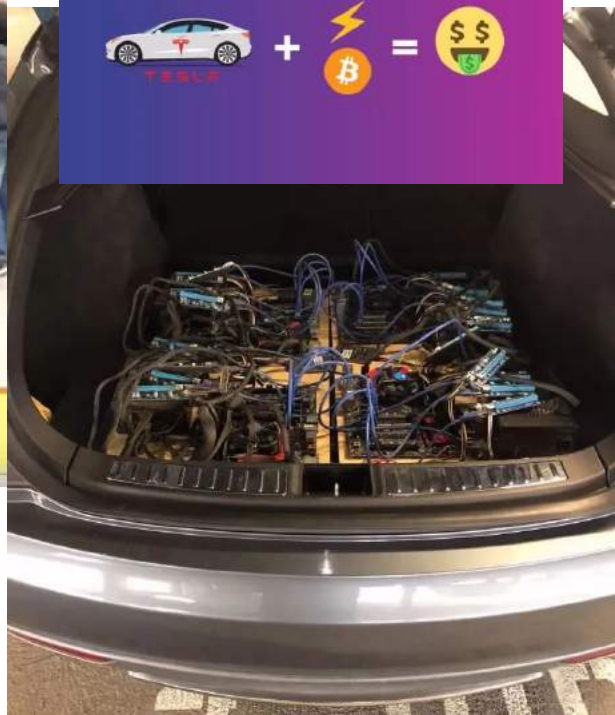
模式动机

- 装饰模式以对客户透明的方式动态地给一个对象附加上更多的责任，客户端并不会觉得对象在装饰前和装饰后有什么不同。装饰模式可以在不需要创造更多子类的情况下，









装饰模式

- 模式定义
  - 装饰模式（Decorator Pattern）动态地给一个对象增加一些额外的职责（Responsibility），就增加对象功能来说，装饰模式比生成子类实现更为灵活
  - 装饰模式也有人称之为“粉刷匠模式”

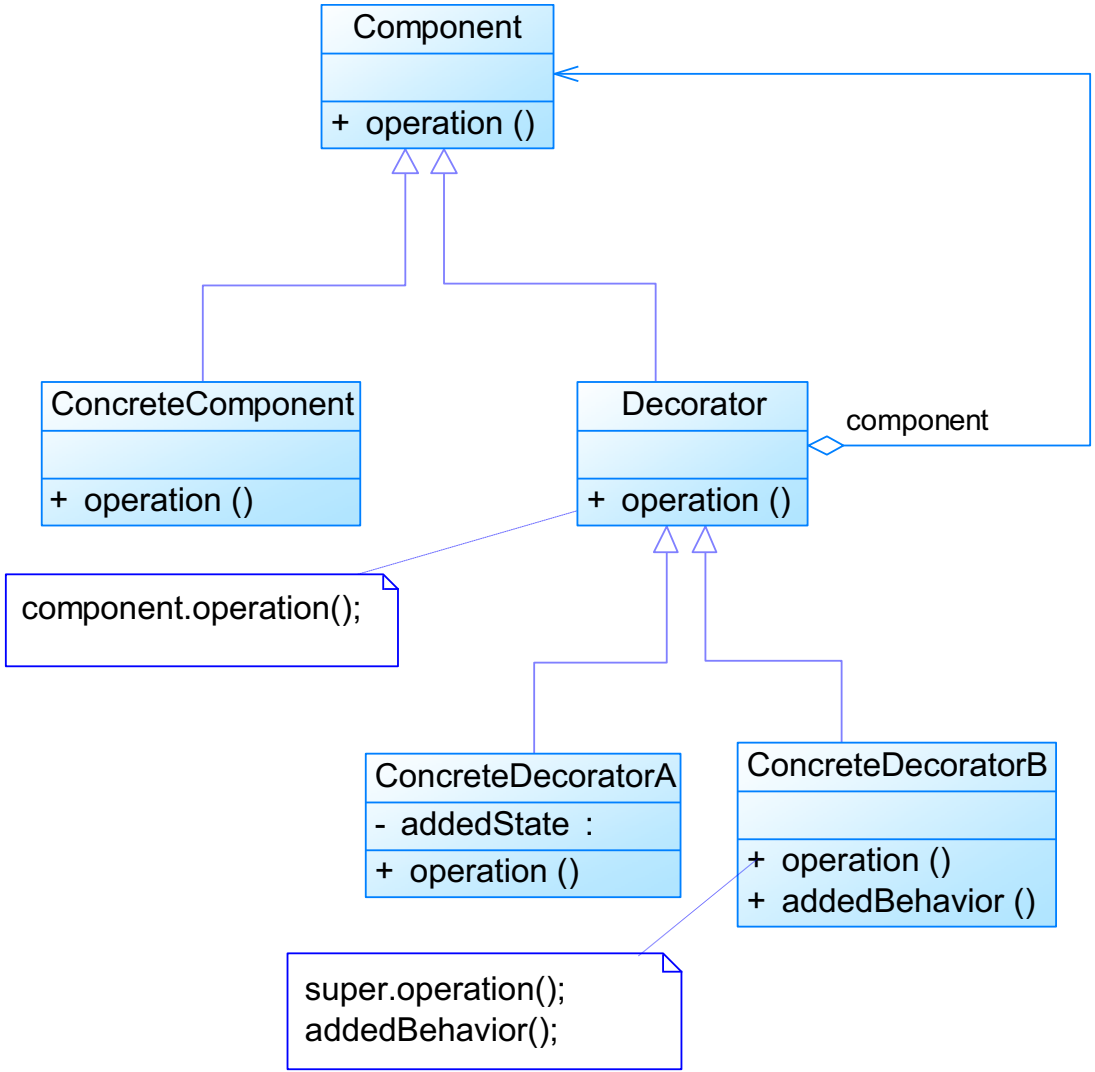


装饰模式

装饰模式包含如下角色：

- Component: 抽象构件
- ConcreteComponent: 具体构件
- Decorator: 抽象装饰类
- ConcreteDecorator: 具体装饰类

模式结构



装饰模式

模式分析

- 与继承关系相比，关联关系的主要优势在于不会破坏类的封装性，而且继承是一种耦合度较大的静态关系，无法在程序运行时动态扩展。在软件开发阶段，关联关系虽然不会比继承关系减少编码量，但是到了软件维护阶段，由于关联关系使系统具有较好的松耦合性，因此使得系统更加容易维护。当然，关联关系的缺点是比继承关系要创建更多的对象。
- 使用装饰模式来实现扩展比继承更加灵活，它以对客户透明的方式动态地给一个对象附加更多的责任。装饰模式可以在不需要创造更多子类的情况下，将对象的功能加以扩展。

装饰模式

- 模式分析
  - 典型的抽象装饰类代码：

```
public class Decorator extends Component {
    private Component component;
    public Decorator(Component component) {
        this.component = component;
    }
    public void operation() {
        component.operation();
    }
}
```

装饰模式

- 模式分析
  - 典型的具体装饰类代码：

```
public class ConcreteDecorator extends Decorator {
    public ConcreteDecorator(Component component) {
        super(component);
    }
    public void operation() {
        super.operation();
        addedBehavior();
    }
    public void addedBehavior() {
        //新增方法
    }
}
```

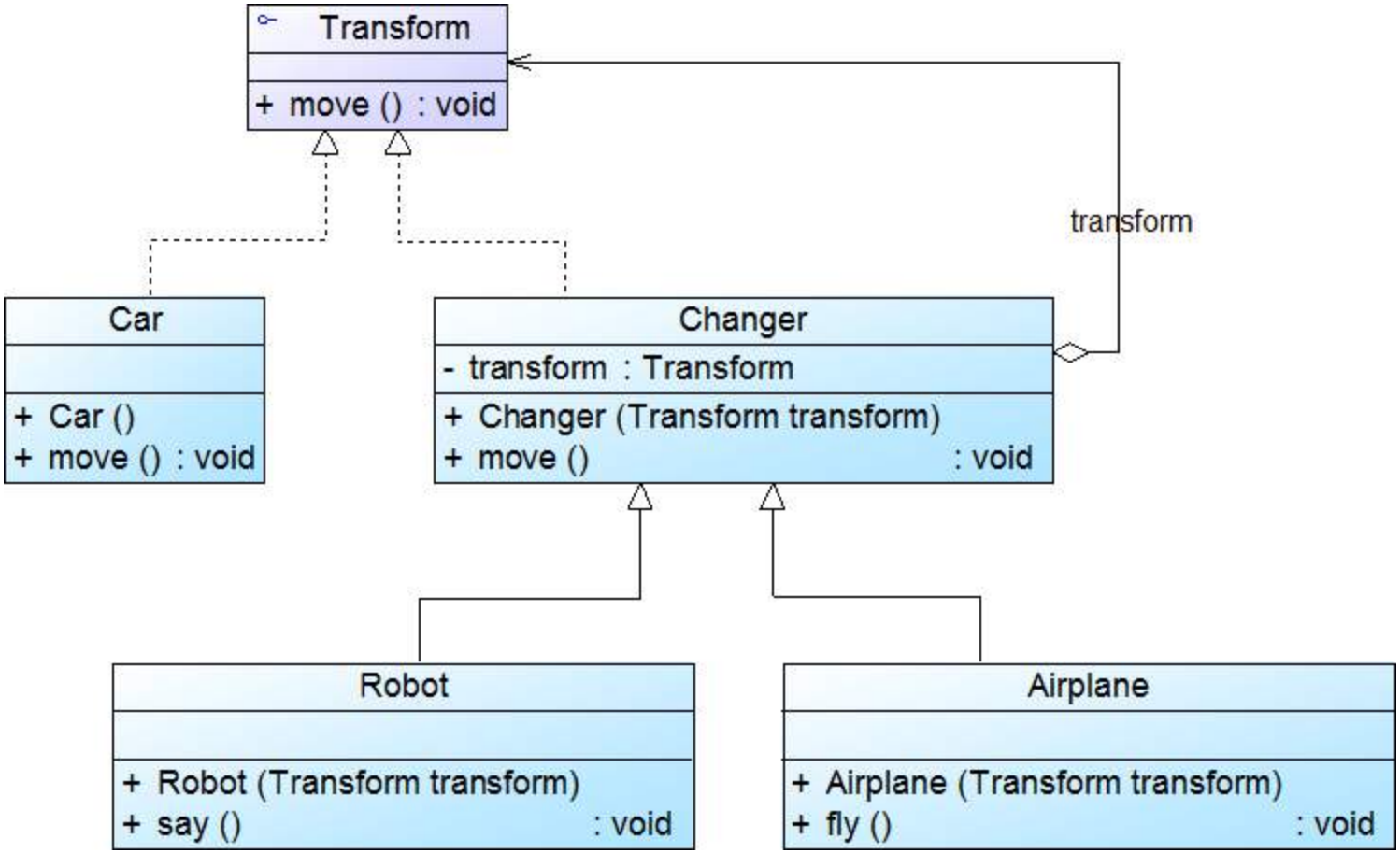
装饰模式

- 装饰模式实例与解析
  - 实例：变形金刚
    - 变形金刚在变形之前是一辆汽车，它可以在陆地上移动。当它变成机器人之后除了能够在陆地上移动之外，还可以说话；如果需要，它还可以变成飞机，除了在陆地上移动还可以在天空中飞翔。



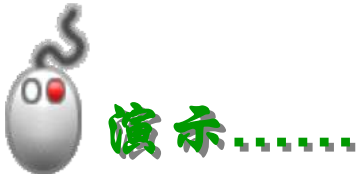
装饰模式

- 装饰模式实例与解析
  - 实例：变形金刚



装饰模式

- 装饰模式实例与解析
  - 实例：变形金刚
    - 参考代码 (decorator)



装饰模式

- 模式优缺点
  - 装饰模式的优点
    - 装饰模式与继承关系的目的都是要扩展对象的功能，但是装饰模式可以提供比继承更多的灵活性
    - 可以通过一种动态的方式来扩展一个对象的功能，通过配置文件可以在运行时选择不同的装饰器，从而实现不同的行为
    - 通过使用不同的具体装饰类以及这些装饰类的排列组合，可以创造出很多不同行为的组合。可以使用多个具体装饰类来装饰同一对象，得到功能更为强大的对象
    - 具体构件类与具体装饰类可以独立变化，用户可以根据需要增加新的具体构件类和具体装饰类，在使用时再对其进行组合，原有代码无须改变，符合“开闭原则”

装饰模式

模式优缺点

装饰模式的缺点

- 使用装饰模式进行系统设计时将产生很多小对象，这些对象的区别在于它们之间相互连接的方式有所不同，而不是它们的类或者属性值有所不同，同时还将产生很多具体装饰类。这些装饰类和小对象的产生将增加系统的复杂度，加大学习与理解的难度。
- 这种比继承更加灵活机动的特性，也同时意味着装饰模式比继承更加易于出错，排错也很困难，对于多次装饰的对象，调试时寻找错误可能需要逐级排查，较为烦琐。

装饰模式

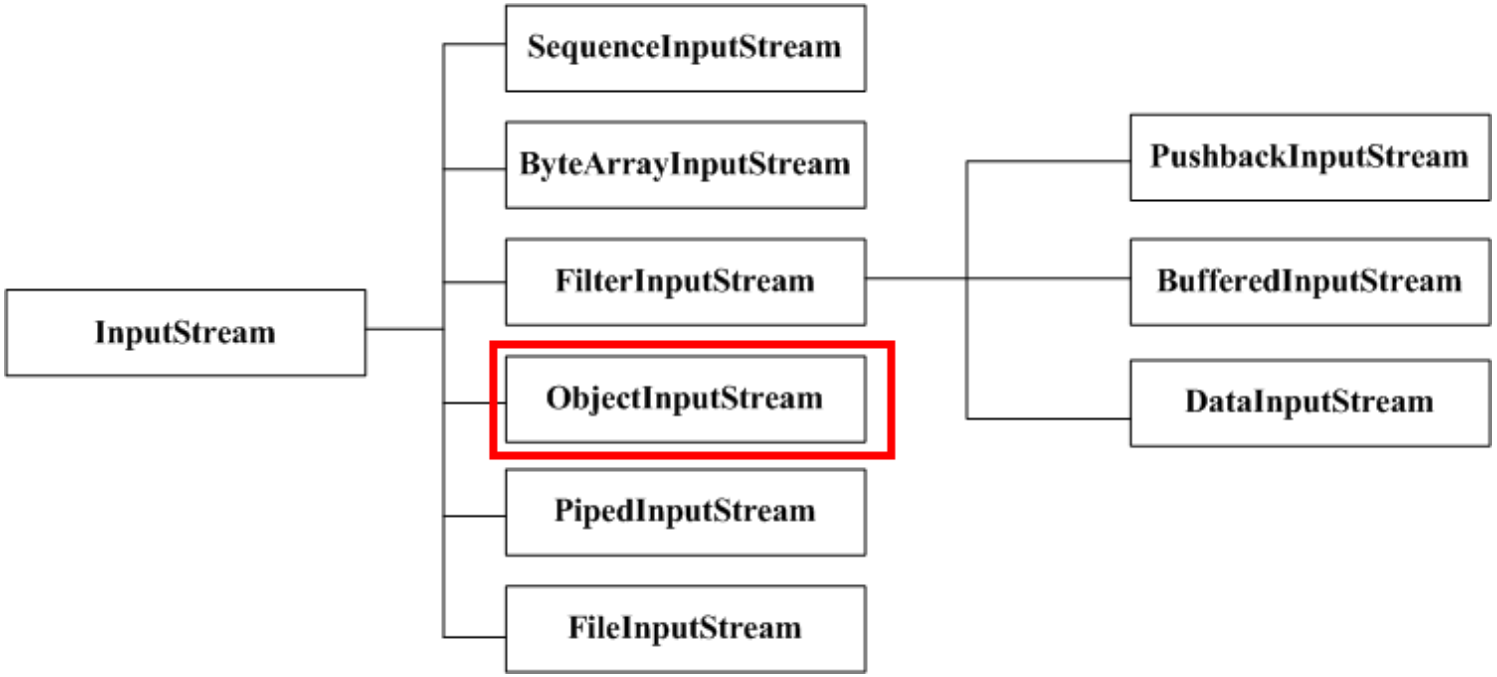
模式应用

- （1）在javax.swing包中，可以通过装饰模式动态给一些构件增加新的行为或改善其外观显示。
  - 如JList构件本身并不支持直接滚动，即没有滚动条，要创建可以滚动的列表，可以使用如下代码实现：

```
JList list = new JList();
JScrollPane sp = new JScrollPane(list);
```

装饰模式

- 模式应用
  - （2）装饰模式在JDK中最经典的实例是Java IO
    - 以InputStream为例



装饰模式

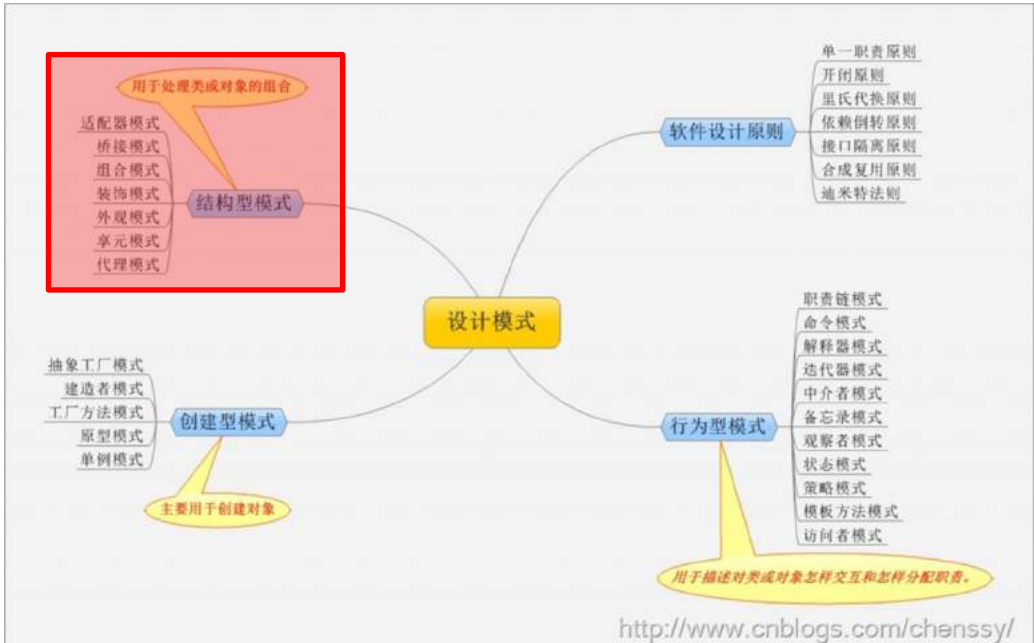
- 模式应用
  - （2）抽象装饰类：FilterInputStream

```
protected volatile InputStream in;
protected FilterInputStream(InputStream in) {
     this.in = in;
}
```

```
FileInputStream inFS = new FileInputStream("temp/fileSrc.txt");
BufferedInputStream inBS = new BufferedInputStream(inFS);
//定义一个字节数组，用于存放缓冲数据
byte[] data = new byte[1024];
inBS.read(data);
```

本章教学内容

- 结构型模式
  - 适配器模式（Adapter）
  - 桥接模式（Bridge）
  - 组合模式（Composite）
  - 装饰模式（Decorator）
  - 外观模式（Facade）
  - 享元模式（Flyweight）
  - 代理模式（Proxy）



第8章

创建型设计模式

陈龙彪
longbiaochen@xmu.edu.cn

厦门大学信息学院
2019年4月